Structural-feature inductive signed-link prediction, and algebraic entropy feedback as a bridge to control

Csaba Hajdu (draft note)

2026-06-18

Abstract

A companion note recording two results and a synthesis. **(1)** The leakage-free "rotor" line's wins are carried by *structural features* replacing the node-identity table, *not* by the rotor's S^3 geometry: a matched-capacity MLP embedding ties the rotor on link-sign AUROC (± 0.002–0.005, inside σ). **(2)** The line's distinctive, falsifiable claim is *inductive*: a model trained on one signed graph transfers to an unseen one, where the transductive (node-table) baselines structurally cannot; on harder train-small/eval-large pairs the learned-from-source increment is positive in 8/8 cells and clears 1σ in 7/8, reaching $+2.90\sigma$ (`bitcoin_otc`→`epinions`). **Synthesis**: the framework's *structural (algebraic) entropy* and the signed-link line's *Lyapunov-safe entropy-feedback* regulariser are the same control idea, and it transfers to reinforcement learning over a robot's kinematic/contact hypergraph — an exploration/regularisation signal driven by the algebra of the state, not by the action distribution alone.

1 The reframe: structural features, not the rotor

The Cayley-rotor embedding replaces the transductive `nn.Embedding(n_nodes)` identity table — the memorisation channel behind the leakage-audit collapse — with a shared map from *structural features* (degree, signed-cycle participation, exact signed A^k walk profile) to a unit quaternion on S^3. The line is leakage-clean (label-shuffle gate → chance) and competitive (deduped strict protocol: 0.850 alpha / 0.879 otc, within $+0.036/+0.016$ of pure SiGAT).

A foundational ablation settles *what* carries the wins. Replacing the rotor embedding with a matched-capacity MLP of equal output dimension (same projection, signed message passing, and readout; $\geq$ parameters) leaves link-sign AUROC unchanged — rotor − MLP $= \pm 0.002$–0.005, inside the seed σ (0.01–0.027), both leakage-clean. The rotor's S^3 geometry is therefore **not load-bearing for AUROC**; the wins are the *structural-feature, leakage-free, inductive* embedding, which the MLP control shares. The residual gap to pure SiGAT is *expressivity* (learned attention), confirmed across input/readout/propagation/depth levers.

2 The distinctive result: inductive transfer

What the rotor (or its MLP sibling) can do that the node-table baselines cannot is *transfer to an unseen graph*: train all weights on graph A, evaluate the frozen model on graph B's strict deduped held-out edges, B's embeddings built from B's own structural features. The transductive control (`dadsgnn`, `nn.Embedding`(n_A)) *cannot index* B — "cannot transfer", recorded, not masked.

To isolate transported knowledge from the eval graph's own structural prior (which signed message passing exposes regardless of training), we decompose each cell into three arms: **real** (train on A), **shuffle** (train on permuted-A signs, the gate), and **random-init** (no training, the floor). Learned transfer = real − max(shuffle, random-init). On the bitcoin pairs the increment

is +0.038–0.063 but only ∼1–1.5σ (suggestive). On harder train-small (bitcoin) → eval-large (slashdot/epinions) pairs it is:

model	pair	real	shuffle	learned	signif.
cayley_rotor	otc→epinions	0.8927	0.8665	+0.0261	$+\mathbf{2.90}\sigma$
cayley_rotor	otc→slashdot	0.8497	0.8282	+0.0215	$+2.28\sigma$
cayley_rotor_walk	otc→epinions	0.8939	0.8418	+0.0520	$+2.02\sigma$
. . . (8 cells)				$8/8 > 0$	$7/8 \geq 1\sigma$

The variance, not the effect size, separates the train graphs: `otc`-trained shuffle arms are tight ($\sigma \approx 0.009 \Rightarrow +2.3$–$2.9\sigma$), `alpha`-trained are noisy ($\sigma \approx 0.05 \Rightarrow \sim 1.1\sigma$ despite a *larger* raw increment). Random-init sits at/below chance throughout, so the cross-graph signal is genuinely *learned*, not the table read trivially.

3 Where the rotor earns its place

Since the AUROC ablation neutralises the rotor, its value lives on a *different* objective: it is weightless (a Cayley map + cross-product sandwich, ∼4 fixed MACs per block, no matrix), and its output is *bounded on* S^3 — calibration-free to quantise and directly cosine/dot-retrievable (ANN-index-ready). So on an embedded / hardware metric the rotor may *win* exactly where it *ties* on AUROC. That is a falsifiable claim (int8 PTQ ΔAUROC, ANN recall@k), reserved for its own test, not asserted here.

4 Algebraic entropy feedback

4.1 Two entropies already in the system

Structural (algebraic) entropy. The compiler computes, tensor-free and deterministically over the hierarchical IR, per-scope Shannon entropies of *structural features only* — arity, sign-roles (+/ − /0), and degree (`hymeko_query::entropy::StructuralEntropy`). This is an algebraic invariant of the hypergraph: how diverse its composition is, independent of any learned weights.

Spectral entropy with Lyapunov-safe KL feedback. The signed-link regulariser acts on the singular-value distribution $\mathbf{p}_t$ of the node-embedding matrix:

$$H_{\text{norm}} = \tfrac{1}{\log_2 \text{rank}} H(\mathbf{p}_t), \quad \Delta_{KL}(t) = D_{KL}(\mathbf{p}_{t-1} \,\|\, \mathbf{p}_t),$$
$$\lambda_{\text{eff}}(t) = \text{clamp}\big(\lambda_0 e^{-\eta \Delta_{KL}(t)},\, 0.1\lambda_0,\, 10\lambda_0\big), \quad \mathcal{L}_H = \lambda_{\text{eff}}\big[\lambda_a (H_{\text{norm}} - H^*)^2 + \lambda_b H_{\text{norm}}\big].$$

The schedule *relaxes* pressure when the spectrum moves fast (transient) and recovers when settled — a feedback loop on the *rate of change* of an entropy. This is already "entropy feedback" (measured: macro-F_1 +0.005 alpha / +0.007 otc).

4.2 The bridge: the same loop, for control

Reinforcement learning maximises $\mathbb{E}[R] + \beta\, H(\pi)$ — an entropy term that buys exploration, but over the *action distribution*, blind to structure. The synthesis: drive the feedback from the **algebraic entropy of the state hypergraph** (the kinematic/contact graph the HSiKAN policy already reads), with the same Lyapunov-safe KL schedule:

- **Signal.** $H_{\text{struct}}(s_t)$ = structural entropy (arity/sign/degree) of the state's hypergraph — high when the contact pattern is rich/novel (a quadruped mid-stride forming closed leg–ground loops), low when degenerate (all feet planted, or airborne).

- **Feedback.** Modulate the exploration/regularisation weight by Δ_{KL} of H_{struct} across steps: relax when the contact structure is churning (the policy is already exploring structurally), tighten when it stalls — the control law transplanted verbatim from $\mathcal{L}_H$.
- **Use.** Either as an intrinsic reward (visit structurally-novel states) or as a structure-aware entropy bonus replacing/augmenting $\beta\, H(\pi)$.

This unifies the two lines: the *same* structural-feature machinery that gives leakage-free inductive signed-link prediction supplies, for free, an algebraic entropy signal whose feedback control is already validated on the signed-link side. It is *proposed*, not measured — the falsifiable test is whether structure-driven entropy feedback improves sample efficiency over the vanilla $\beta\, H(\pi)$ bonus on the `hymeko_rl` arm/quadruped tasks, ablated against the MLP policy (which has no hypergraph to read an algebraic entropy from).

5 Open questions

- **Which entropy, where.** H_{struct} is of the *fixed* topology for a rigid robot; the time-varying part is the *contact* hypergraph (transient foot–ground edges). The feedback signal must read the contact graph, not just the kinematic skeleton — ties to the legged contact-mode line.
- **Confound control.** Any RL gain must be attributed to the algebraic entropy, not extra reward shaping: fix the PPO algorithm, ablate only the entropy source.
- **Significance of the alpha-trained transfer cells.** A 10-seed pass would settle whether the noisy $\sim 1.1\sigma$ cells also clear 1σ.

Provenance. Reports `2026-06-18-{rotor-vs-mlp-embed-ablation, inductive-transfer-test, transfer-grid-strengthen}.md`; regulariser in `paper/regulariser_note.tex` § spectral-entropy; entropy compiler in `hymeko_query::entropy`; RL substrate in `hymeko_rl/`.